

Microprocesadores y Microcontroladores

PRUEBA DE EVALUACIÓN CONTINUA 1

Programación de microcontroladores

GRADO EN INGENIERÍA ELECTRÓNICA INDUSTRIAL Y AUTOMÁTICA
GRADO EN INGENIERÍA ELÉCTRICA.



¿Cómo realizar esta práctica?

Esta práctica consiste en varios ejercicios guiados que el alumno deberá completar utilizando la herramienta Tinkercad: <http://www.tinkercad.com>.

Una vez completados cada uno de esos ejercicios guiados el alumno deberá realizar las modificaciones propuestas sobre esos diseños, tanto a nivel hardware como software.

Entregable: El alumno enviará a través de la tarea correspondiente en el curso virtual una memoria de las prácticas en formato PDF que contenga tanto el **código fuente** cargado en Arduino, un **pantallazo del diseño**, una **explicación textual del código original**, así como de cada una de las **modificaciones propuestas** y el **enlace al circuito generado en Tinkercad** de manera que el Equipo Docente pueda acceder y ejecutar la simulación. Dichas simulaciones deberán funcionar correctamente de acuerdo a cada una de las modificaciones propuestas. Se deben respetar los colores de los cables para facilitar su revisión (rojo para 5V y negro para tierra).

¿Cómo empezar a trabajar con Tinkercad?

Tinkercad es una aplicación web que permite crear proyectos de Arduino con layout de protoboard. Los pasos a dar son los siguientes. Primero, accede a <http://www.tinkercad.com> y regístrate como nuevo usuario (si no lo eres ya).



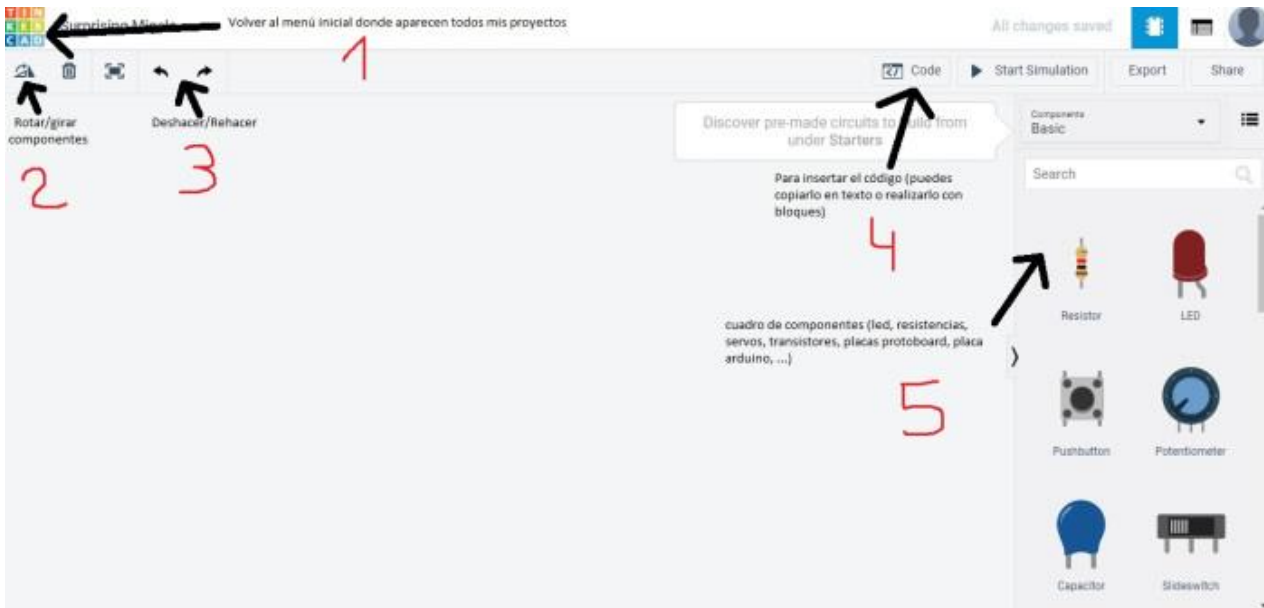
Una vez registrados accedemos a la pantalla donde podemos ver los circuitos que hemos ido creando. En este caso estará vacía. Debemos pulsar en CIRCUITS y clickamos en crear nuevo circuito para comenzar a montar y programar nuestros circuitos digitales. Aquí te explica para que sirven algunas de las acciones.



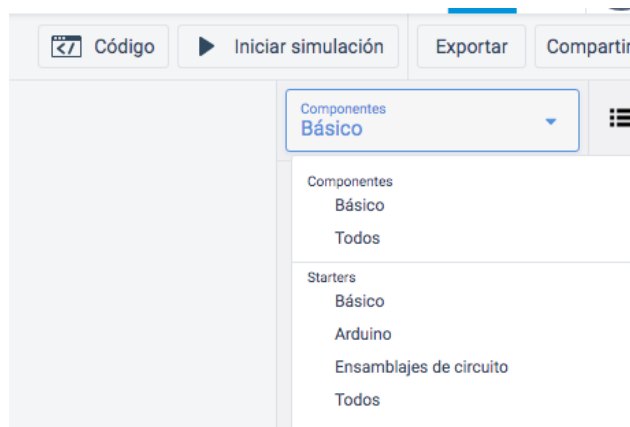
Una vez creado nuestro nuevo circuito merece la pena revisar los principales menús, botones y pestañas:

1. Volver al menú inicial donde aparecen todos mis proyectos

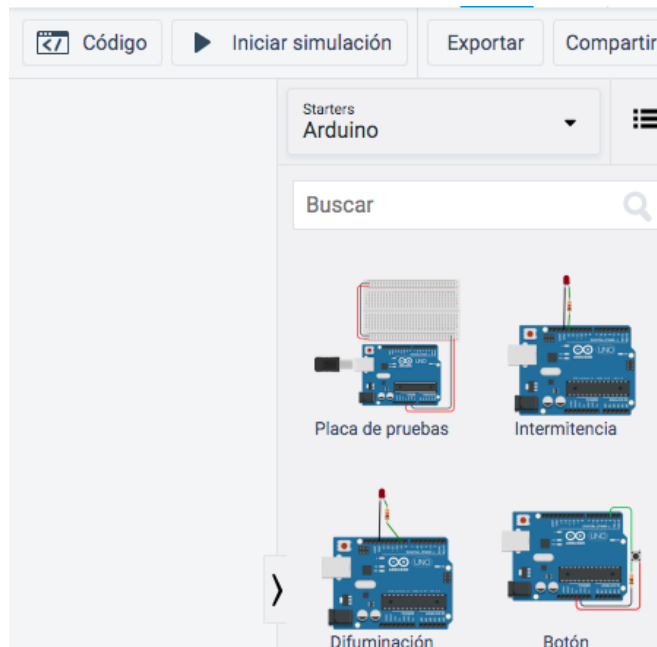
2. Rotar/Girar componentes (sobre todo válido para resistencias)
3. Deshacer o rehacer la última acción realizada
4. Para insertar el código (si ya lo tienes lo puedes copiar en "texto" sino lo puedes hacer por bloques)
5. Cuadro de componentes electrónicos (hay una opción en la que se visualizan solamente los básicos y otra opción para que aparezcan todos los componentes posibles).
6. Tienes que saber que al hacer clic en algunos componentes te da la opción de cambiar sus características, por ejemplo el color de los LED, el color del cable o los ohmios en las resistencias.



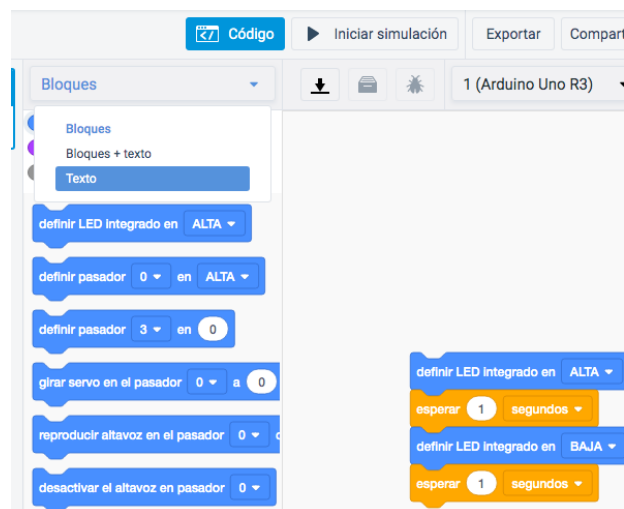
Si pulsamos en el desplegable Componentes podemos seleccionar la opción Starters Arduino.



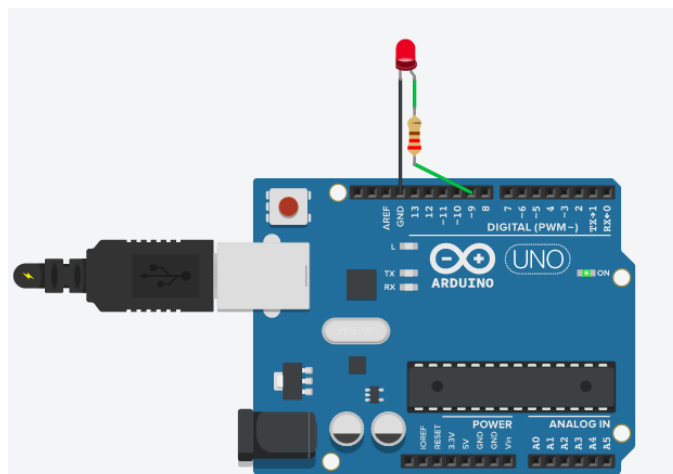
Al hacerlo nos aparecerán numerosos circuitos premontados y preconfigurados con un código fuente de ejemplo para probar dicho diseño.



Se recomienda arrastrar uno de ellos al lienzo central y pulsar en el botón Código para inspeccionar el código de ejemplo que viene por defecto. Dicho código de ejemplo puede visualizarse en modo programación de bloques, en modo bloques+texto o en modo texto. En nuestro caso seleccionaremos modo texto para tener control total de la programación a ejecutar en la placa.



Una vez inspeccionado el código y entendido cómo funciona se puede pulsar el botón Iniciar simulación. A continuación se simulará el funcionamiento de la ejecución del código en dicho circuito.



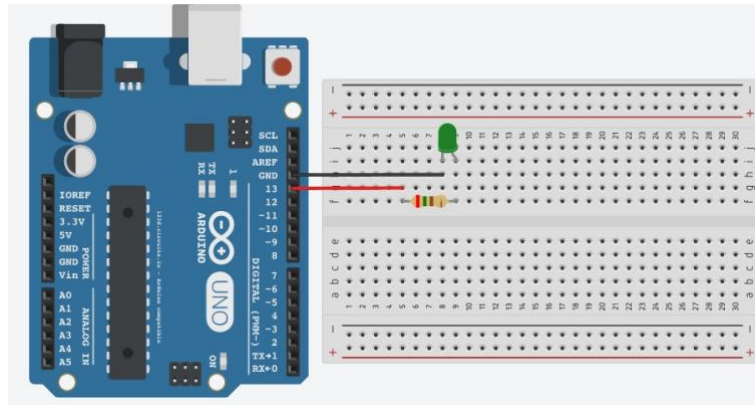
Práctica 1. Parpadeo de LED

1.1 Descripción

En este proyecto haremos que se ilumine un LED en modo intermitente, de esta forma, al cambiar su estado entre alto y bajo alternativamente lo veremos parpadear continuamente.

Para este esquema solo necesitaremos una salida digital, por lo que, como podemos ver, será el puerto 13, que suministrará energía intermitente al LED.

1.2 Esquema



1.3 Principales bloques

En este circuito simple, el componente principal es el LED que configuramos, por ejemplo, en el puerto digital 13 en modo de salida y una resistencia de $220\text{K}\Omega$. A continuación establecemos un valor para la velocidad de parpadeo, con el comando "delay()", en este ejemplo se ha elegido medio segundo (500 ms), disminuyendo este valor veremos que el parpadeo gira más rápido. También vamos a hacer uso del monitor serie para tener un log de lo que va ocurriendo, al cual se accede desde la pestaña Código, en la parte inferior "Monitor serie".

1.3 ¿Qué hacer?

Monta en Tinkercad un circuito como el de la figura anterior, y en la sección de código carga el siguiente código para ver su ejecución.

1.4 Código

```
const byte pin_led = 13;

void setup()
{
  Serial.begin(9600);           // Serial port speed: 9600 symbols per second
  pinMode(pin_led, OUTPUT);     // Digital port 13 as output
}

void loop()
{
  digitalWrite(pin_led, HIGH); // turn the LED "on" (HIGH is the voltage level)
  Serial.println("On");        // Print on serial monitor "On" state, and cursor goes to new line (\n)
  delay(500);                  // wait during 500 ms
  digitalWrite(pin_led, LOW);  // turn the LED "off" (LOW will be the voltage level)
  Serial.println("Off");       // Print on serial monitor "Off" state, and cursor goes to new line (\n)
  delay(500);                  // wait during a 500 ms
}
```

1.5 Tareas a realizar

1. Describe cómo se está utilizando el monitor serie y la función delay.
2. Incluye en tu diseño 2 LEDs más, de manera que tengas uno rojo, uno naranja y uno verde.
3. Prográmalos de manera que construyas un semáforo. El funcionamiento será cíclico: primero se encenderá el

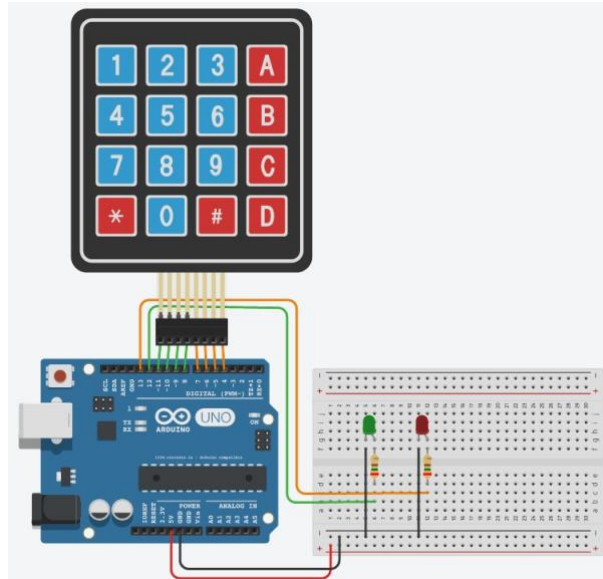
rojo durante tres segundos, luego el verde durante otros 3 segundos y finalmente el naranja durante 500ms.

Práctica 2. Control de acceso con Keypad 4x4

2.1 Descripción

En este nuevo esquema realizaremos un circuito capaz de controlar el acceso de entrada a una puerta. Con este teclado 4x4 tendremos la posibilidad de almacenar cualquier contraseña tecleada, que será procesada y comparada con una contraseña registrada válida, para esta tarea, utilizaremos una contraseña de 4 números, que será: 1234

2.2 Esquema

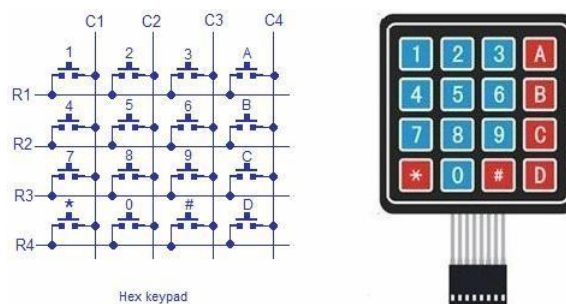


2.3 Principales bloques

El componente principal de este control de diseño es el teclado 4x4, que presenta una forma cuadrada con 4 filas y 4 columnas. Además incluimos en el diseño un LED verde y uno rojo, que se iluminarán si se concede acceso o no respectivamente. Además haremos uso del monitor en serie.

El funcionamiento interno es simple, como se puede ver en la figura a continuación, se colocan 4 filas perpendicularmente con 4 columnas, de modo que, cada vez que presione cualquier tecla, se conectarán dos líneas y el microcontrolador sabrá qué carácter se presionó.

Esta información viene dada por el bus serie de 8 pines unido al teclado. Los primeros cuatro a la izquierda se usan para filas y las cuatro líneas en la parte superior derecha estarán destinadas a columnas.



2.4 Código

```
//***** KEYPAD CONFIGURATION *****
#include <Keypad.h> //Keypad header file. Necessary to execute the functions below

const byte ROWS = 4; //number of rows
const byte COLS = 4; //number of columns
const byte green_pin_led = 12; //our green led is connected to pin 13
const byte red_pin_led = 13; //our red led is connected to pin 13

//keypad character matrix
char keys[ROWS][COLS] =
{
  { '1', '2', '3', 'A' },
  { '4', '5', '6', 'B' },
  { '7', '8', '9', 'C' },
  { '*', '0', '#', 'D' },
};

byte rowPins[ROWS]={11,10,9,8}; // digital pins connected to rows
byte colPins[COLS]={7,6,5,4}; // digital pins connected to columns

//setting all the parameters of the keypad
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

//***** END KEYPAD CONFIGURATION *****

void setup()
{
  pinMode(red_pin_led, OUTPUT); //red led
  digitalWrite(green_pin_led, LOW); //green led off
  digitalWrite(red_pin_led, LOW); //red led off
}

int i=0;
char valid [5] = { '1', '2', '3', '4', '\0' }; //PASSWORD to access
char pass [5] = { '1', '2', '3', '4', '\0' }; //matrix to store the PIN
char result [5] = { '1', '2', '3', '4', '\0' }; //result matrix. 4 characteres = '1' (ACCESS AUTHORIZED)

void loop()
{
  char key = keypad.getKey(); //pressed key will be stored in key
  if (key != NO_KEY) //if any key is pressed...
  {
    pass[i]=key; //store the pressed keys along the 'pass' array

    if(pass[i]==valid[i]) //if stored position is equal to the same position of valid password...
    {
      result[i]='1'; // '1' will be stored in result
    }
    else
    {
      result[i]='0'; // '0' will be stored in result
    }

    i++; // Once the 4 numbers are stored, (positions 0 to 3), i=4 and next if, will be executed

    if (i==4)
    {
      i=0;

      if(result[0]=='1' && result[1]=='1' && result[2]=='1' && result[3]=='1') //if PIN is correct...
      {
        delay(300);
        digitalWrite(green_pin_led, HIGH); //green led will light during 2s
        delay(2000);
        digitalWrite(green_pin_led, LOW);
      }
      else //if PIN is not correct...
      {
        digitalWrite(red_pin_led, HIGH); //red led will flash twice
        delay(300);
        digitalWrite(red_pin_led, LOW);
        delay(300);
        digitalWrite(red_pin_led, HIGH);
        delay(300);
        digitalWrite(red_pin_led, LOW);
      }
    }
  }
}
```

2.5 Referencias

How to use a keypad 4x4 with Arduino: https://www.youtube.com/watch?v=_loTs_9Ltio

2.6 Tareas a realizar

1. Describe cuál es el funcionamiento del teclado 4x4
2. Describe cómo se lee el dato del teclado y se compara con la contraseña válida almacenada.
3. Modifica el código de manera que si se introducen 3 veces seguidas una contraseña errónea, el sistema no admita nuevas contraseñas hasta pasados 10 segundos.
4. Cambiar el cableado y el código de manera que las filas estén conectadas a los pines 7,6,5 y 4, y las columnas a los pines 11, 10, 9 y 8, en lugar de como las tienes ahora.
5. Modifica el código para que se escriba por el monitor serie el número introducido y se especifique cuando se otorga o deniega acceso.

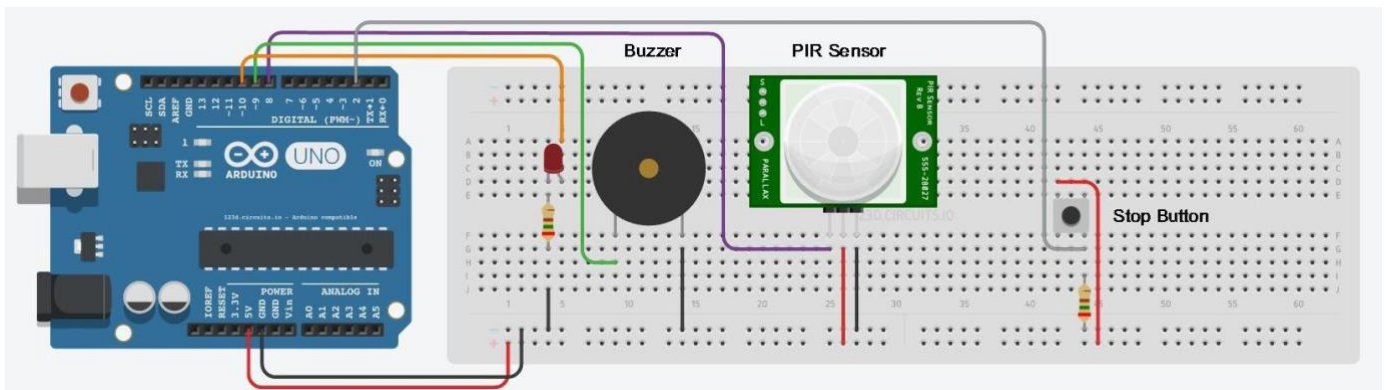
Práctica 3. Detector de presencia con alarma sonora y visual

3.1 Descripción

En este proyecto, crearemos un circuito capaz de detectar cualquier movimiento dentro de un área específica, que se muestra en verde claro una vez iniciada la simulación.

El detector verificará cualquier movimiento interno, continuamente, y enviará dos posibles valores lógicos al pin digital 8: '0' No se detectó movimiento y '1' Movimiento detectado. En este segundo caso, una alarma estará activa continuamente hasta que alguien presione el botón de parada.

3.2 Esquema



3.3 Principales bloques

El componente eléctrico principal de este diseño es el detector pasivo de infrarrojos, también llamado sensor PIR. Este dispositivo es capaz de medir la energía que produce la temperatura superior al cero absoluto, esta energía no es visible para los humanos porque funciona en la longitud del infrarrojo con grandes longitudes de onda, pero será detectable para los sensores PIR.

El sensor PIR se mantendrá continuamente comprobando la energía recibida desde el área cubierta, si esta energía aumenta (al introducir cualquier objeto dentro del área) o disminuye (al salir de algo de esta área), el sensor del PIR enviará un 'H' digital de alto nivel. al puerto conectado, y obviamente activando cualquier señal de advertencia o alarma.

En el caso de movimiento detectado, un zumbador y un led rojo se activarán intermitentemente configurando dos estados diferentes, un estado alto con un zumbido de 600 Hz durante 0.6s y led, 'encendido' y un estado bajo con un zumbido de 300 Hz ruido durante 0.2s y led, 'apagado'.

3.4 Código

```
int input=0;
int alarm=0;

const byte pin_led = 10;
const byte pin_sensor = 8;
const byte pin_buzzer = 9;
const byte pin_pulsador = 2;

void setup()
{
  Serial.begin(9600); //serial port speed: 9600 symbols per second
  pinMode(pin_sensor, INPUT);
  pinMode(pin_buzzer, OUTPUT);
  pinMode(pin_led, OUTPUT);
  pinMode(pin_pulsador, INPUT); //interrupt input. 2 possibilities for ArduinoUno [pin2-INT0] or [pin3-INT1]
                                //we will use the first one to execute the interrupt function

  attachInterrupt(0, stop, RISING); //interrupt instruction
                                    //[0(input from pin 2), function 'stop', active by rising edge
}

void loop()
{
  input = digitalRead(pin_sensor); //store digital input from pin 8 in 'input'
  serial.println(input);           //input value on Serial Monitor: '0' NO MOTION DETECTED - '1' MOTION DETECTED

  if(input == HIGH) //if motion detected...
  {
    alarm =1; //alarm will be set in a high mode statically, until 'stop' button will be pressed
  }

  Serial.print("alarm: ");
  Serial.println(alarm); //alarm state on Serial Monitor

  if (alarm==1)
  {
    tone(pin_buzzer, 600, 600); //this function will define the parameters to the buzzer for create
                                //the alarm (output by pin 9, buzz a signal with 600 Hz, during 0.6s)
    digitalWrite(pin_led, HIGH); //led 'on'
    delay(600);                  //keep two instructions of above active during 0.6s

    tone(pin_buzzer, 300, 200); //(output by pin 9, 300Hz, 0.2s)
    digitalWrite(pin_led, LOW);  //led 'off'
    delay(200);                  //keep two instructions of above active during 0.2s
  }
}

void stop()
{
  digitalWrite(pin_buzzer, LOW); //buzzer 'off'
  digitalWrite(pin_led, LOW);    //led 'off'
  alarm=0;
}
```

3.5 Referencias

- <http://cursoarduinomega.blogspot.com.es/2015/04/sonido-de-alarma-buzzer-tmb12a05.html>
- <https://www.arduino.cc/reference/en/language/functions/advanced-io/tone/>
- <https://www.luisllamas.es/detector-de-movimiento-con-arduino-y-sensor-pir/>

3.6 Tareas a realizar

1. Describe cómo funciona y se usa el pulsador.
2. Describe cómo funciona el buzzer.
3. Añade al circuito también un motor de vibración, de manera que cuando salte la alarma se active también el motor de vibración. Su funcionamiento es simple, el



terminal negativo va a tierra y el positivo a un pin digital de Arduino.

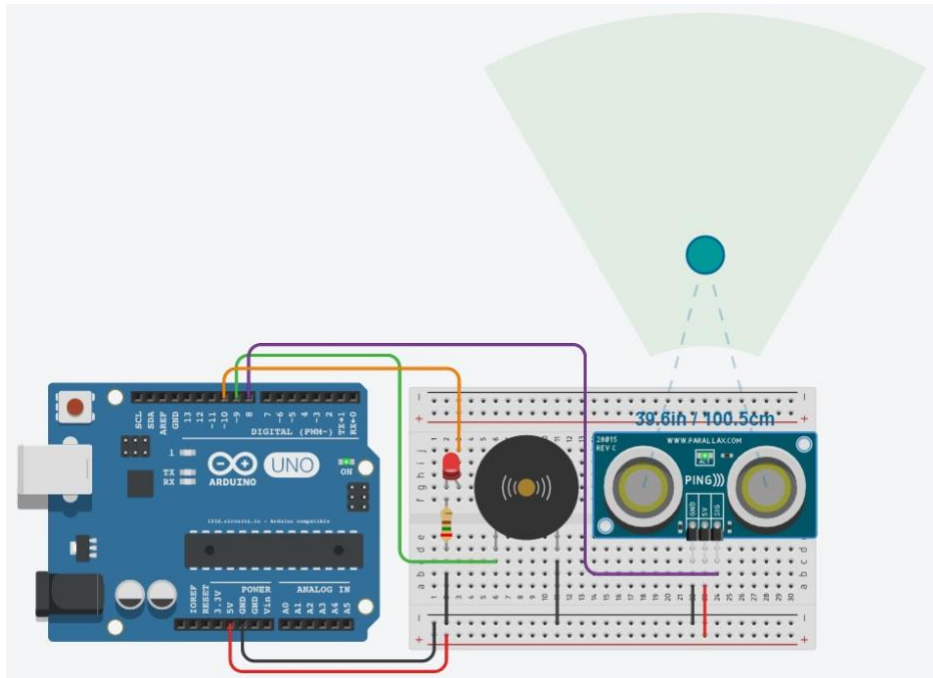
Práctica 4. Detector de obstáculos con alarma intermitente

4.1 Descripción

En este nuevo proyecto crearemos un Esquema, capaz de detectar cualquier obstáculo dentro del área cubierta por el detector ultrasónico.

La distancia entre el detector y el obstáculo se calculará y registrará, y dependiendo de esta longitud, los pitidos emitidos por el timbre se repetirán más rápido, acercándose la distancia entre ellos.

4.2 Esquema



4.3 Principales bloques

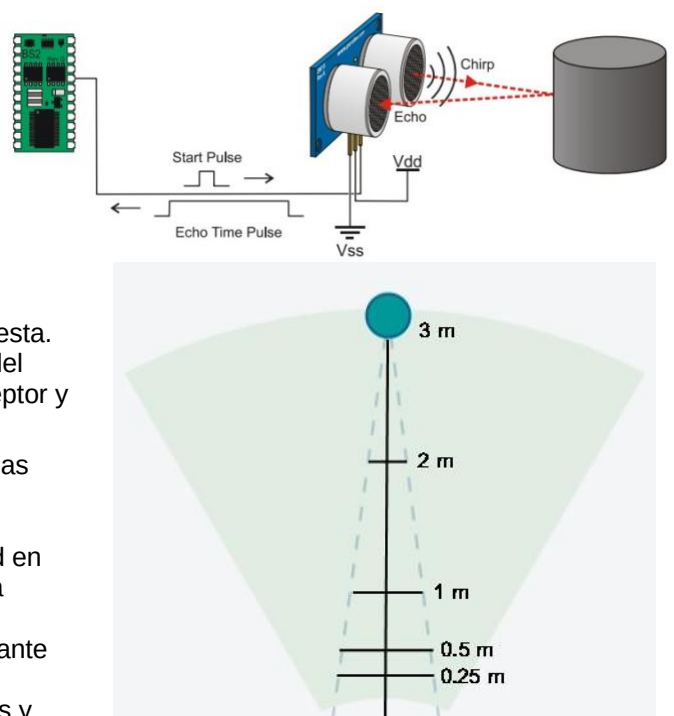
El componente principal de este Esquema, es el detector ultrasónico, y su método de trabajo es el siguiente, por su puerto conectado, cuyo comportamiento será de entrada o salida, dependiendo del momento.

En una primera situación, le proporcionaremos una señal de pulso, esta señal de ping será emitida por el emisor, este pulso se reencontrará y el receptor lo obtendrá.

Después de este proceso, se registrará el tiempo de respuesta. Una vez en este punto, y teniendo en cuenta la velocidad del sonido (340 m/s) podemos calcular la longitud entre el receptor y el obstáculo.

Como podemos ver en la hoja de datos de este elemento, las mediciones de distancia que puede detectar provienen de aproximadamente 2 cm (0,8 pulgadas) a 3 metros (118,11 pulgadas). Para nuestro proyecto, dividiremos esta longitud en cinco secciones diferentes, como puede ver en la imagen a continuación.

El pitido para la alerta se crea con una señal de 400Hz durante 0.1s, y dependiendo de la posición, donde se encuentra el obstáculo, aumentaremos el tiempo de silencio entre pitidos y pitidos, siendo 0.3s, para el rango 3 - 2m, 0.2 s para el rango 2 - 1m, 0.1s para el rango 1 - 0.5m, 0.05s para el rango de 0.5 - 0.25, y finalmente un pitido continuo para la distancia 0.25m y 0.20m (distancia más baja alcanzable para este sensor).



4.4 Código

```
const byte pin_led = 10;
const byte pin_sensor = 8;
const byte pin_buzzer = 9;

void setup()
{
  Serial.begin(9600); //serial port speed: 9600 symbols per second
  pinMode(pin_led, OUTPUT); //led pin
}

void loop()
{
  float duration, inch, cm;

  pinMode(pin_sensor, OUTPUT); //creating the ping pulse 'LOW' 2us, 'HIGH' 5us
  digitalWrite(pin_sensor, LOW);
  delayMicroseconds(2);
  digitalWrite(pin_sensor, HIGH);
  delayMicroseconds(5);
  digitalWrite(pin_sensor, LOW);
  pinMode(pin_sensor, INPUT); //After sampling ping pulse, pin 8 as input

  duration = pulseIn(pin_sensor, HIGH); //response time for getting back the ping pulse [microseconds]

  cm = duration / 29 / 2; //sound speed: 340 m/s or 29 microseconds per centimeter
  inch = duration / 74 / 2; //sound speed: 13385.83 inches/s or 74 microseconds per inch

  Serial.print(duration);
  Serial.println(" microseconds");
  Serial.print(inch);
  Serial.print(" in");
  Serial.println();
  Serial.print(cm);
  Serial.println(" cm");
  Serial.println();

  delay(100);

  if(cm<=300 && cm>200) //range 3m - 2m [func4]
    func4();
  if(cm<=200 && cm>100) //range 2m - 2m [func3]
    func3();
  if(cm<=100 && cm>50) //range 1m - 0.5m [func2]
    func2();
  if(cm<=50 && cm>25) //range 0.5m - 0.25m [func1]
    func1();
  if(cm<=25) //range < 0.25m (continuous beep) [func0]
    func0();
}

void func4()
{
  tone(pin_buzzer, 400, 100); // (pin9, 400Hz, 100ms) led will be blinking
  digitalWrite(pin_led, HIGH);
  delay(100);
  noTone(pin_buzzer);
  digitalWrite(pin_led, LOW);
  delay(300); //between beep and beep, silence during 300ms
}

void func3()
{
  tone(pin_buzzer, 400, 100);
  digitalWrite(pin_led, HIGH);
  delay(100);
  noTone(pin_buzzer);
  digitalWrite(pin_led, LOW);
  delay(200); //between beep and beep, silence during 200ms
}

void func2()
{
  tone(pin_buzzer, 400, 100);
  digitalWrite(pin_led, HIGH);
  delay(100);
  noTone(pin_buzzer);
  digitalWrite(pin_led, LOW);
  delay(100); //between beep and beep, silence during 100ms
}

void func1()
{
  tone(pin_buzzer, 400, 100);
  digitalWrite(pin_led, HIGH);
  delay(100);
}
```

```

    noTone(pin_buzzer);
    digitalWrite(pin_led, LOW);
    delay(50); //between beep and beep, silence during 50ms
}

void func0()
{
    tone(pin_buzzer, 400); //continuous beep and led 'on'
    digitalWrite(pin_led, HIGH);
}

```

4.5 Referencias

<http://mick-st.blogspot.com.es/2012/05/medidor-de-distancias-por-ultrasonidos.html>

<https://www.parallax.com/sites/default/files/downloads/28015-PING-Documentation-v1.6.pdf>

<https://www.arduino.cc/en/Reference/PulseIn>

<https://create.arduino.cc/projecthub/muhammad-aqib/arduino-rgb-led-tutorial-fc003e>

4.6 Tareas a realizar

1. Describe el funcionamiento, uso y programación del sensor de ultrasonidos usado en esta práctica.
2. Cambia el LED rojo por uno RGB, de manera que según esté en una de las 5 zonas de proximidad cambie de color (por ejemplo: verde, amarillo, naranja, magenta, rojo). Se puede aprender a usar el LED RGB en este enlace: <https://create.arduino.cc/projecthub/muhammad-aqib/arduino-rgb-led-tutorial-fc003e>
3. Describe cómo se programa el LED RGB.

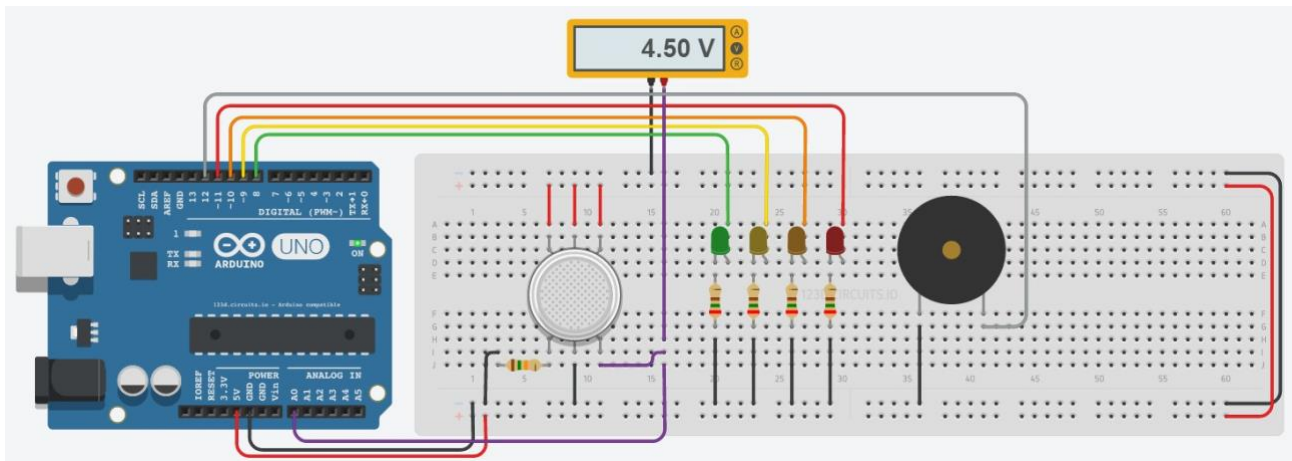
Práctica 5. Detector de gas con alarma

5.1 Descripción

En este proyecto crearemos un diseño que nos proporcionará información sobre la calidad del aire, pudiendo detectar gases inflamables, humo y otros gases peligrosos.

Estableceremos 4 rangos de medición diferentes, siendo el primero la situación de mayor seguridad, subiendo a un led verde y pasando por el siguiente con un led amarillo, y el siguiente con un led naranja que se muestra, finalmente llegamos al máximo situación peligrosa donde el gas peligroso presenta su densidad máxima, en este caso sonará una alarma y se mostrará un led rojo.

5.2 Esquema



5.3 Principales bloques

El sensor de gas analógico MQ2 es capaz de detectar gas LP (licuado petróleo), butano, propano, metano, alcohol, hidrógeno, tiene una alta sensibilidad y un tiempo de respuesta rápido. Este pequeño sensor de gas detecta la presencia de gases inflamables y humo, en concentraciones de 300 a 10,000 ppm.

Incorpora una interfaz de voltaje analógica simple, que solo requiere estar conectada a un pin de entrada analógica en el microcontrolador.



Al conectar 5V en el sensor, los pines de la fuente de alimentación podrán mantenerse lo suficientemente calientes como para funcionar correctamente. Simplemente conecte 5v a cualquier pin (A o B) para obtener un valor de voltaje. Finalmente, la sensibilidad del detector se establece con una resistencia de carga entre los pines de salida y tierra.

Después de esta configuración, y obteniendo el voltaje de salida de una carga de resistencia de 10k Ω , obtenemos el valor mínimo de entrada: 590 (2.88v) y un valor máximo de 921 (4.50v) para la densidad máxima de detección de gas. Dividiendo este rango en 3 áreas iguales, obtenemos la siguiente tabla:

Posición	Valor entrada	Voltios entrada
1	590 - 699	Min. 2.88v
2	700 - 809	
3	810 - 921	Max. 4.50v

Así, en nuestra práctica asignaremos un color de LED a cada una de estas zonas (amarillo, naranja y rojo). Adicionalmente tendremos en cuenta que si el valor de entrada es menor que 590 (ausencia de contaminantes) iluminaremos el led verde.

5.4 Código

```
int input=0; //analog value will be stored here
const byte pin_led_verde=8;
const byte pin_led_amarillo=9;
const byte pin_led_naranja=10;
const byte pin_led_rojo=11;
const byte pin_buzzer=12;

void setup()
{
  Serial.begin(9600); //serial port speed: 9600 symbols per second

  pinMode(pin_led_verde, OUTPUT); //green led
  pinMode(pin_led_amarillo, OUTPUT); //yellow led
  pinMode(pin_led_naranja, OUTPUT); //orange led
  pinMode(pin_led_rojo, OUTPUT); //red led
  pinMode(pin_buzzer, OUTPUT); //Buzzer
}

void loop()
{
  input=analogRead(A0); //input value
  Serial.println(input); //Show this value on Serial Monitor

  if (input<590) //1st range situation. The lowest density
  {
    digitalWrite(pin_led_verde, HIGH); //green led
    'on'
    digitalWrite(pin_led_amarillo, LOW);
    digitalWrite(pin_led_naranja, LOW);
    digitalWrite(pin_led_rojo, LOW);
    noTone(pin_buzzer);
  }

  if (input>=590 && input<700) //2nd range situation
  {
    digitalWrite(pin_led_verde, LOW);
    digitalWrite(pin_led_amarillo, HIGH); //yellow
    led 'on'
    digitalWrite(pin_led_naranja, LOW);
    digitalWrite(pin_led_rojo, LOW);
    noTone(pin_buzzer);
  }

  if (input>=700 && input<810) //2nd range situation
  {
    digitalWrite(pin_led_verde, LOW);
    digitalWrite(pin_led_amarillo, LOW);
    digitalWrite(pin_led_naranja, HIGH); //orange led 'on'
    digitalWrite(pin_led_rojo, LOW);
    tone(pin_buzzer, 600); //buzzer beeps continuously
  }

  if (input>=810) //4th range situation. The highest density
  {
    digitalWrite(pin_led_verde, LOW);
    digitalWrite(pin_led_amarillo, LOW);
    digitalWrite(pin_led_naranja, LOW);
    digitalWrite(pin_led_rojo, HIGH); //red led 'on'
    tone(pin_buzzer, 300); //buzzer beeps continuously
  }
}
```

5.5 Referencias

<http://hetpro-store.com/TUTORIALES/sensor-de-gas-mq2/>
https://create.arduino.cc/projecthub/Aritro/smoke-detection-using-mq-2-gas-sensor-79c54a?ref=tag&ref_id=monitoring&offset=18

5.6 Tareas a realizar

1. Describe el funcionamiento, uso y programación del sensor de gas.
2. Añade el componente microservo a un pin digital que controle una trampilla de ventilación. Cuando se esté en la zona amarilla el servo se situará en la posición 30 (de 180). Cuando esté en la zona naranja se situará en la posición 90, y cuando esté en la zona roja se situará en la posición 180. Podrás leer cómo usarlo en la página oficial de la librería Servo: <https://www.arduino.cc/en/reference/servo>. Básicamente lo que tienes que hacer es importar la librería servo.h en tu proyecto, crear un objeto tipo Servo, en el setup indicar en qué pin lo tienes conectado (función attach) y finalmente indicar la posición en la que quieres que se situe (write).
3. Describe el funcionamiento, uso y programación del microservo.

